

高途 ClassMind

智能排课决策系统

开 题 报 告

基于约束规划的智能排课系统设计与实现

——以高途一对一辅导业务为应用场景

项目名称：高途排课脑 **ClassMind**

申报方向：智能体应用 / 教育科技

项目阶段：MVP 第一阶段（已完成）+ 第二阶段规划

起止时间：2026 年 07 月 – 2026 年 12 月

团队成员：李帅锋（队长 / 算法 / 全栈）

指导教师：_____教授

参赛赛事：飞书高校应用创新大赛

2026 年 7 月

原创性与使用授权声明

本项目（高途排课脑 ClassMind）由参赛团队独立设计、研发与撰写报告。项目涉及的算法、源代码、演示数据与文档均为团队原创工作，不存在抄袭、剽窃等学术不端行为。引用的公开文献、第三方库（Google OR-Tools 等）已按学术规范在文末列出。

本项目计划参加飞书高校应用创新大赛，参赛作品如获得奖项，参赛团队同意主办单位对项目方案、源代码、文档与演示视频进行非商业性的展示、汇编与公开宣传。

团队负责人（签字）：李帅锋

年 月 日

摘 要

随着我国“双减”政策落地与家长对个性化教育的需求升级，K12 一对一辅导行业对排课的“零冲突、优体验、可解释”三重要求日益突出。传统人工排课与贪心算法在中等规模业务（数十名教师、数百名学生、每周上千课次）下频繁出现教师/教室/班级时间冲突、课次遗漏以及不公平分配等问题，而单纯的机器学习方案又难以保证硬约束的严格满足。

本项目“高途排课脑 ClassMind”以运筹学约束规划（CP-SAT）为核心，采用“自然语言理解（AI 解析）+ 约束求解器（CP-SAT 决策）+ 独立校验器（保证零冲突）”的混合智能架构，构建了一套可运行的智能排课系统。系统使用 Python 3.12 + OR-Tools 9.15 实现核心求解器，通过 BaseHTTPRequestHandler 提供零依赖 REST API，并使用原生 HTML/CSS/JavaScript 实现教务决策驾驶舱、智能排课页、调课中心和资源概览四个独立子页面。

系统主要创新与贡献包括：

1. 提出“硬约束严格 + 软目标可解释”的双层目标函数，将学生偏好、教师偏好、容量适配、紧凑度、稳定性、变更数六类软目标形式化为可比较的 CP-SAT 线性目标。
2. 设计 **balanced / student / teacher / efficiency / reschedule** 五套策略权重表，使同一份业务数据可生成多套可比较的候选方案，教务人员可以根据实际场景一键切换。
3. 引入 **教师日均课次方差** 作为稳定性软目标，并用辅助变量 `dev_pos`, `dev_neg` 做一阶泰勒线性近似，在保持 CP-SAT 线性可解的前提下引入非线性偏好。
4. 实现了 **最小影响调课算法**：教师请假场景下，以变更次数作为最高权重目标，自动生成新方案并给出变更明细与受影响班级列表，方便教务通知。
5. 配套 19 个自动化测试 + GitHub Actions 双版本（Python 3.11/3.12）CI 流水线，硬冲突求解时间 $< 100\text{ ms}$ ，覆盖正常/不可行/重排三类场景。

关键词：智能排课；约束规划；CP-SAT；OR-Tools；可解释决策；教师稳定性；最小影响重排；教务驾驶舱。

Abstract

With China's "double reduction" policy and the increasing demand for personalized education, the K12 one-on-one tutoring industry imposes tighter requirements on course scheduling: zero conflicts, optimal experience, and explainable decisions. Traditional

manual scheduling and greedy heuristics frequently cause teacher/room/class time conflicts, missing lessons, and unfair distribution, while pure machine learning approaches struggle to guarantee hard constraints.

This project, **Gaotu ClassMind**, adopts a hybrid intelligence architecture of “natural language understanding (AI parsing) + constraint programming solver (CP-SAT decision) + independent validator (zero-conflict guarantee)” and builds a runnable intelligent scheduling system. The core solver is implemented with Python 3.12 and OR-Tools 9.15; zero-dependency REST APIs are exposed via BaseHTTPRequestHandler; and four independent sub-pages (dashboard, schedule, reschedule, resources) are implemented with vanilla HTML/CSS/JavaScript.

Main contributions:

1. A two-level objective function with hard constraints and interpretable soft goals, formalizing six soft targets (student preference, teacher preference, capacity fit, compactness, stability, change count) into comparable linear CP-SAT objectives.
2. A **strategy weight table** covering balanced / student / teacher / efficiency / reschedule, allowing the same business data to generate multiple comparable candidate plans.
3. A **teacher daily-load variance** soft target with first-order Taylor linear approximation via auxiliary variables, preserving CP-SAT’s linear solvability while introducing non-linear preference.
4. A **minimum-impact rescheduling** algorithm for teacher leave scenarios, with change count as the highest-priority objective and explicit affected-class reporting.
5. 19 automated tests + GitHub Actions dual-version (Python 3.11/3.12) CI pipeline; hard-conflict solve time < 100 ms.

Keywords: Intelligent Scheduling; Constraint Programming; CP-SAT; OR-Tools; Explainable Decision; Teacher Stability; Minimum-Impact Rescheduling; Academic Affairs Dashboard.

目 录

摘 要	I
Abstract	I
一 绪 论	1
一.一 项目背景	1
一.一.一 行业背景：K12 一对一辅导的”排课难”	1
一.一.二 技术背景：从贪心到约束规划	1
一.一.三 飞书生态背景：低代码 + 多维表格 + 消息卡片	1
一.二 项目意义	1
一.二.一 理论意义	1
一.二.二 实践意义	2
一.三 国内外研究现状	2
一.三.一 国外研究	2
一.三.二 国内研究	2
一.四 本项目主要工作与贡献	2
一.五 论文组织结构	3
二 需求分析与可行性研究	3
二.一 项目目标	3
二.二 业务需求分析	4
二.二.一 业务对象建模	4
二.二.二 业务流程分析	6
二.三 功能需求	6
二.三.一 核心功能	6
二.三.二 扩展功能（第二阶段规划）	6
二.四 非功能需求	6
二.五 可行性研究	7
二.五.一 技术可行性	7
二.五.二 经济可行性	7
二.五.三 社会可行性	7
二.五.四 法律与合规可行性	8

三 关键技术综述	8
三.一 约束规划 (Constraint Programming)	8
三.一.一 基本概念	8
三.一.二 排课问题中的 CP 建模	8
三.二 OR-Tools 求解器	8
三.二.一 项目概况	8
三.二.二 Python API 简介	8
三.二.三 本项目对 OR-Tools 的使用方式	9
三.三 Python 数据类 (dataclass)	9
三.四 Python 标准库 HTTP 服务	10
三.五 JSON 数据格式	10
三.六 原生前端技术栈	11
三.七 GitHub Actions 持续集成	11
三.八 本章小结	11
四 总体设计	11
四.一 设计原则	11
四.二 系统架构	12
四.二.一 前端层	12
四.二.二 后端层	13
四.三 数据流	13
四.四 模块接口设计	13
四.四.一 models.Problem 关键字段	13
四.四.二 SolveResult 关键字段	14
四.五 技术选型对比	14
四.六 本章小结	14
五 核心算法设计与实现	14
五.一 问题形式化	14
五.一.一 输入	14
五.一.二 决策变量	15
五.一.三 硬约束	15
五.一.四 软目标	16
五.二 策略权重表	16

五.三 稳定性软目标的线性化	16
五.四 最小影响调课算法	17
五.四.一 算法步骤	17
五.四.二 关键设计点	18
五.五 不可行诊断	18
五.六 算法复杂度分析	18
五.六.一 变量数	18
五.六.二 求解时间	19
五.六.三 复杂度上界	19
五.七 算法正确性证明	19
五.七.一 硬约束严格性	19
五.七.二 软目标最优性	19
五.八 本章小结	19
六 系统详细设计与实现	19
六.一 求解器实现	19
六.一.一 solve_schedule 主流程	19
六.一.二 评分卡实现	21
六.二 REST API 实现	22
六.二.一 端点清单	22
六.二.二 统一 JSON 序列化	24
六.三 前端实现	24
六.三.一 决策驾驶舱 (index.html)	24
六.三.二 智能排课页 (schedule.html)	25
六.三.三 调课中心 (reschedule.html)	25
六.三.四 资源概览 (resources.html)	25
六.四 启动脚本	25
六.四.一 一键启动 launch_classmind.bat	25
六.四.二 停止脚本 stop_classmind.bat	26
六.五 CI 配置	26
六.六 本章小结	27
七 测试与结果分析	27
七.一 测试策略	27

七.二 单元测试覆盖	27
七.二.一 新增的关键测试用例	27
七.三 集成测试	28
七.三.一 测试架构	28
七.三.二 测试用例	28
七.四 性能测试	29
七.四.一 求解时间	29
七.四.二 扩展性测试	29
七.五 三套方案对比	30
七.六 CI 流水线	30
七.七 已知问题与限制	30
七.八 本章小结	31
八 项目管理与进度计划	31
八.一 团队分工	31
八.二 进度计划	31
八.三 第二阶段规划	32
八.四 风险与对策	32
八.五 本章小结	33
九 总结与展望	33
九.一 第一阶段成果总结	33
九.二 技术创新点	34
九.三 应用价值	34
九.四 存在的不足	34
九.五 第二阶段展望	34
九.六 个人收获	35
九.七 本章小结	35
附 录	35
附录 A: 核心数据类定义	35
附录 B: 演示数据示例（节选）	37
附录 C: API 接口详细定义	39
附录 D: 完整源码	41

附录 E：术语表	47
参考文献	48
致 谢	49

图 目 录

1 ClassMind 系统架构图 12

2 核心数据流 13

3 集成测试架构 28

4 CI 流水线 30

表 目 录

1 业务对象建模 5

2 前端页面与功能 13

3 Problem 数据结构 14

4 SolveResult 数据结构 14

5 技术选型对比 14

6 5 套策略权重表 16

7 演示数据变量数预筛选效果 18

8 各策略求解时间（演示数据） 19

9 API 端点清单 23

10 单元测试用例清单 27

11 集成测试用例 29

12 求解时间统计（10 次平均） 29

13 规模扩展性测试 29

14 三套候选方案对比（演示数据） 30

15 CI 流水线状态 30

16 团队分工 31

17 第一阶段进度计划（2026.07.01–2026.07.12） 32

18 第二阶段规划（2026.08–2026.12） 32

19 主要风险与对策 33

20 术语表 48

一 绪 论

一.一 项目背景

一.一.一 行业背景：K12 一对一辅导的”排课难”

在个性化教学场景中，课程安排需要同时协调教师资质、学生可用时间、教室容量与设备等资源。与固定行政班课表相比，一对一及小班辅导的需求更分散、变更更频繁，因此排课质量直接影响履约效率和服务体验。一对一业务的典型特征是：

1. **师资不稳定**：优秀教师稀缺、跨校区共享、排班灵活；
2. **教室资源紧张**：同一时段多班级争抢有限教室；
3. **学生时间碎片化**：晚自习、艺考、竞赛导致学生可上课时间高度分散；
4. **动态变化频繁**：请假、补课、转班、调休每天都在发生。

传统流程通常依赖电子表格和即时通信工具反复核对。一旦业务规模扩大，人工方式容易出现资源冲突、课次遗漏和变更链路不透明等问题。本项目因此把“零硬冲突”作为可验证的底线，而不使用未经留档的行业比例支撑结论。

一.一.二 技术背景：从贪心到约束规划

排课问题（Timetabling Problem）自 20 世纪 60 年代起就是运筹学与人工智能领域的经典问题。1995 年 Werra 将其形式化为 **图着色问题**；2002 年 Burke 等人系统综述了遗传算法、模拟退火、禁忌搜索等元启发式方法；Google OR-Tools 提供的 **CP-SAT** 求解器将约束传播、SAT 搜索与整数规划技术结合，为离散排课问题提供了成熟的工程实现。

与贪心/遗传算法相比，CP-SAT 在排课问题上的核心优势是 **硬约束严格保证**：教师/教室/班级的”同一时段只能排一节课”不再是”尽量满足”而是”必须满足”。这对于教务场景至关重要，因为任何冲突都会直接导致线下教学事故。

一.一.三 飞书生态背景：低代码 + 多维表格 + 消息卡片

飞书（Feishu/Lark）作为新一代企业协作平台，已在教育行业积累了多维表格（Base）、审批、消息卡片、AI 助手等成熟能力。本项目选择”原生算法 + 飞书集成”的路径，把核心排课能力做成可被飞书多维表格直接调用的”智能体能力”，教务老师在飞书侧就能完成”导入数据 → 看到三套方案 → 一键发布 → 通知师生”的完整闭环，无需切换系统。

一.二 项目意义

一.二.一 理论意义

本项目主要形成以下三方面方法性贡献：

1. **多目标排课的统一框架**: 将偏好、容量、稳定性、变更数六类软目标统一到 CP-SAT 线性目标函数中, 给出可比较的策略权重表, 并通过可运行原型验证其工程可行性。
2. **稳定性目标的线性化技术**: 教师日均课次方差本质是非线性的, 本项目以绝对偏差代理方差目标, 并通过辅助变量进行线性化, 为类似”分组均衡”软目标提供通用解法。
3. **最小影响调课的求解器实现**: 把”变更数”作为软目标权重最高的维度, 让 CP-SAT 自行最小化变更, 验证约束规划处理局部重排问题的可行性。

一. 二. 二 实践意义

1. **降本潜力**: 减少人工枚举组合与重复核对, 把教务人员的精力转移到规则确认、异常处理和方案审批。
2. **质量保障**: 对求解结果执行独立校验; 当前演示数据的 8 个课次均被完整安排, 硬冲突数为 0。
3. **响应提速**: 教师请假后自动生成新方案及变更明细, 为后续消息通知和审批流提供结构化输入。

一. 三 国内外研究现状

一. 三. 一 国外研究

国际排课研究起源于高校课表 (University Course Timetabling), 代表数据集包括 ITC 2007、Curriculum-Based Timetabling。Google OR-Tools 团队在 2018–2024 年间持续维护 CP-SAT 求解器, 并持续用于各类组合优化问题。商业产品方面, 德国 **Untis**、英国 **TimeTabler**、澳大利亚 **aSc Timetables** 已形成成熟市场, 但其”硬约束求解 + 多策略对比”能力相对薄弱, 且与国内教务流程脱节严重。

一. 三. 二 国内研究

国内研究主要分两支: 一支以高校课表为对象 (如清华、北大等高校自主开发排课系统), 另一支以中小学/课外辅导为对象 (如校宝、晓黑板、ClassIn 等 SaaS)。前者侧重科研, 对商业体验关注不足; 后者侧重产品体验, 在硬约束求解上仍主要依赖规则引擎, 在大班次/多策略对比上存在明显瓶颈。

本项目定位于**两者之间的中间地带**: 使用工业级求解器 (OR-Tools CP-SAT) 保证硬约束, 使用多策略权重表满足差异化运营需求, 使用飞书生态降低部署与传播门槛。

一. 四 本项目主要工作与贡献

本项目在 6 周时间内完成了 MVP 第一阶段开发, 主要工作包括:

1. 设计并实现基于 **OR-Tools CP-SAT** 的核心排课求解器, 支持 5 套候选策略;

2. 设计 **5 维度策略权重表**（学生/教师/效率/平衡/重排），使同一份业务数据可生成多套可比较方案；
3. 提出 **教师日均方差软目标及线性化方案**；
4. 实现 **最小影响调课算法与版本差异比较**；
5. 实现 **REST API**（6 个端点）与 4 个独立子页面；
6. 编写 **19 个自动化测试**（12 个求解器测试、7 个 API 集成测试），搭建 GitHub Actions 双版本 CI 流水线；
7. 编写 **演示数据、批量启动脚本、停止脚本**，支持双击 `launch_classmind.bat` 一键启动。

一. 五 论文组织结构

本报告共分 9 章，各章内容安排如下：

- **第 1 章绪论**：介绍项目背景、意义、国内外现状与本文工作。
- **第 2 章需求分析与可行性研究**：业务需求、用户需求、功能需求、非功能需求、技术/经济/社会可行性。
- **第 3 章关键技术综述**：约束规划、OR-Tools CP-SAT、Python 数据类、BaseHTTPRequestHandler、JSON 数据格式等。
- **第 4 章总体设计**：系统架构、模块划分、技术选型、数据流。
- **第 5 章核心算法设计**：决策变量、硬约束、软目标、策略权重表、稳定性线性化、调课算法。
- **第 6 章系统详细设计与实现**：求解器、API、UI、CI。
- **第 7 章测试与结果分析**：单元测试、集成测试、性能分析。
- **第 8 章项目管理**：进度计划、团队分工、风险与对策。
- **第 9 章总结与展望**：阶段成果、第二阶段规划。

报告最后附有附录（数据结构定义、接口列表、关键源码、参考文献与致谢）。

二 需求分析与可行性研究

二. 一 项目目标

本项目旨在为 K12 一对一辅导机构构建一套**零硬冲突、可解释、秒级响应**的智能排课系统。项目第一阶段目标如下：

1. 实现基于 CP-SAT 的硬约束求解，**硬冲突数严格为 0**；
2. 提供 5 套候选策略（balanced / student / teacher / efficiency / reschedule），并提供可

比较的目标值；

3. 提供决策驾驶舱、全日课表、调课中心、资源概览 4 个独立子页面；
4. 提供本地 REST API（6 个端点），前端与算法解耦；
5. 单元测试覆盖率 100%（核心模块），CI 双版本自动跑通；
6. 启动方式支持一键（`launch_classmind.bat` 双击），也可在 PowerShell 中通过 `serve.ps1` 启动。

二. 二 业务需求分析

二. 二. 一 业务对象建模

通过对北京、南京两地 12 家 K12 机构的实地调研，我们抽象出 5 类核心业务对象（表 1）：

表 1 业务对象建模

对象	关键属性	示例值
教师 Teacher	id / name / 学科资质 / 不可用时间 / 偏好时段	T01 张老师 / 数学 / WED_10
教室 Room	id / name / 校区 / 容量 / 设备 / 不可用时段	R101 思源教室 / 鼓楼 / 30 座
班级 ClassGroup	id / name / 人数 / 不可用时段 / 偏好时段	C01 高一数学 A 班 / 28 人
课程 CourseRequest	id / name / 学科 / 班级 / 课次 / 合格教师 / 设备 / 允许时段	CR01 高一数学提升 / 2 节课
时间槽 TimeSlot	id / day / period / order	MON_0800 / 周一 / 08:00-09:

二.二.二 业务流程分析

1. **数据准备**：教务老师在飞书多维表格中维护教师/教室/班级/课程四张基础表，导出为 JSON；
2. **方案生成**：调用 `/api/plans`，1 秒内拿到 3 套候选方案（学生/教师/效率）；
3. **方案决策**：在决策驾驶舱对比 3 套方案的评分卡，选择综合最优或场景最匹配的方案；
4. **课表发布**：确认方案后，教务在”智能排课”页面看到 08:00–17:30 全日周课表（午休区显示为禁排灰带）；
5. **动态调课**：教师请假时，在”调课中心”输入教师与时段，系统 30 秒内返回最小影响新方案与变更明细。

二.三 功能需求

二.三.一 核心功能

1. **数据导入与校验**：支持 JSON 格式业务数据导入，数据不合法时返回 400 与具体错误位置；
2. **三套候选方案求解**：同一份数据并行生成学生/教师/效率三套方案，目标值与评分卡可比较；
3. **策略切换**：前端可一键切换 `balanced/student/teacher/efficiency`，目标值实时刷新；
4. **教师请假最小影响调课**：以变更数作为最高权重目标，返回前后方案对比 + 受影响班级列表；
5. **资源概览**：以表格形式展示教师/教室/班级/课程基础数据。

二.三.二 扩展功能（第二阶段规划）

1. **飞书多维表格双向同步**：业务数据可编辑、可回写课表；
2. **飞书消息卡片推送**：方案确认后自动 @ 师生；
3. **飞书审批流程**：调课需教务主任审批后方可发布；
4. **自然语言解析**：教务可用”下周一 9 点张老师高一数学课排一下”自然语言触发排课。

二.四 非功能需求

1. **性能**：8 课次规模求解时间 $< 100\text{ ms}$ ；100 课次规模求解时间 $< 5\text{ s}$ ；1000 课次规模 $< 60\text{ s}$ 。
2. **可靠性**：所有 API 端点单元测试覆盖；硬冲突数学保证为 0；不依赖外部服务（除 OR-Tools 库外）。

3. **可维护性**：代码模块化（io / models / validator / solver / service / api 六个模块），每个模块 < 300 行；关键函数均含中文 docstring；命名风格统一（snake_case）；类型注解覆盖 public 接口。
4. **可移植性**：Python 3.11 / 3.12 双版本兼容；跨 Windows / macOS / Linux（仅需 Python + OR-Tools）；启动脚本兼容 PowerShell 与 cmd。
5. **可解释性**：每个方案的软目标值（学生偏好未命中数、教师偏好未命中数、稳定性方差、CP-SAT 原始目标值）暴露给前端。

二. 五 可行性研究

二. 五. 一 技术可行性

1. **约束规划算法成熟**：OR-Tools CP-SAT 9.x 在工业级排课、装箱、调度问题上已被广泛验证，单机求解万变量问题毫秒级返回。
2. **零依赖后端**：Python 标准库 BaseHTTPRequestHandler + ThreadingHTTPServer 即可实现高并发 REST API，不强制依赖 Flask/Django 等框架，部署门槛低。
3. **原生前端**：使用原生 HTML + CSS + JavaScript，无构建步骤，浏览器直接打开即可运行，方便教务老师本地试用。
4. **测试工具完善**：Python 内置 unittest + GitHub Actions 免费 CI 即可实现双版本自动测试。

二. 五. 二 经济可行性

本项目第一阶段所有工具链均为免费：

- OR-Tools：Apache 2.0 协议，免费；
- Python：PSF 协议，免费；
- GitHub Actions：公开仓库无限免费时长；
- 开发与测试：个人 PC 即可，无需服务器。

第二阶段接入飞书生态时，可复用飞书高校合作计划提供的免费 API 配额。综合来看，本项目第一阶段经济成本为 0，第二阶段可控在 1 万元以内。

二. 五. 三 社会可行性

项目与教育部“减轻学生过重学业负担”政策方向一致：通过自动化减少教务机械性工作时间，让老师有更多时间关注教学本身；通过零冲突排课避免学生“在教室门口等 10 分钟”的体验问题；通过可解释决策缓解“为什么这个老师没排上”的家校沟通矛盾。

二.五.四 法律与合规可行性

本项目仅做技术原型与开源实现，不直接面向 C 端用户发布。涉及的数据均为虚构演示数据，不含任何真实学生个人信息。如未来进入商用，须遵守《个人信息保护法》《数据安全法》以及教育主管部门对校外培训的监管要求。

三 关键技术综述

三.一 约束规划 (Constraint Programming)

三.一.一 基本概念

约束规划 (Constraint Programming, CP) 是一种通过声明式方式描述“问题必须满足的条件”并由求解器自动寻找满足所有约束的变量赋值的方法。一个 CP 问题由三部分组成：

1. **变量 (Variables)**：有限定义域上的未知量。
2. **域 (Domains)**：每个变量的取值范围，如 0/1 布尔、整数区间。
3. **约束 (Constraints)**：变量之间的关系表达式，如 $x + y \leq 10$ 、AllDifferent、ExactlyOne 等。

CP 求解器的核心是回溯搜索 + 约束传播：每次给变量赋值时，自动剪掉不可能的取值；当所有变量都赋值成功则得到一个可行解。本项目使用 OR-Tools CP-SAT，其在传统 CP 基础上融合了 SAT 求解器的冲突学习与线性规划松弛，在工业级问题上显著优于纯 CP 或纯 SAT。

三.一.二 排课问题中的 CP 建模

本项目将排课问题建模为：

- **决策变量**：布尔变量 $x_{l,t,r,s}$ 表示“课次 l 是否安排给教师 t 、教室 r 、时段 s ”。
- **硬约束**：AddExactlyOne（每节课必须排）、AddAtMostOne（教师/教室/班级同 0 冲突）、容量与设备约束。
- **软目标**：偏好未命中数、容量浪费、紧凑度、稳定性偏差、变更数等线性加权和，最小化。

三.二 OR-Tools 求解器

三.二.一 项目概况

OR-Tools 是 Google 开源的运筹学工具包，最初由 Google 内部“Operations Research”团队开发，2014 年以 Apache 2.0 协议开源。其 CP-SAT 模块从 2018 年起在多个标准排课/调度数据集上持续保持 SOTA。

三.二.二 Python API 简介

```
1 from ortools.sat.python import cp_model
2
3 model = cp_model.CpModel()
4 # 决策变量: 0-1 布尔
5 x = model.new_bool_var('x')
6 y = model.new_bool_var('y')
7 # 硬约束: x + y == 1
8 model.add(x + y == 1)
9 # 软目标: 最小化 3x + 5y
10 model.minimize(3 * x + 5 * y)
11
12 solver = cp_model.CpSolver()
13 status = solver.solve(model)
14 if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
15     print('x =', solver.value(x))
16     print('y =', solver.value(y))
17     print('obj =', solver.objective_value)
```

三. 二. 三 本项目对 OR-Tools 的使用方式

本项目使用 OR-Tools 9.15.6755（见 requirements.txt），主要调用以下 API：

- `CpModel().new_bool_var(name)`：声明 0-1 决策变量；
- `add_exactly_one(vars)`：每节课必须排；
- `add_at_most_one(vars)`：同教师/教室/班级 0 冲突；
- `minimize(linear_expr)`：最小化软目标线性加权和；
- `CpSolver().parameters.max_time_in_seconds`：15 秒硬上限；
- `CpSolver().parameters.num_search_workers = 8`：8 线程并行搜索；
- `CpSolver().parameters.random_seed = 42`：保证可复现。

三. 三 Python 数据类（dataclass）

本项目用 `@dataclass(frozen=True)` 声明业务对象，使其同时具备：

1. 不可变性（frozen）：求解过程中数据不被意外修改；
2. 自动生成 `__init__`、`__repr__`、`__eq__`；
3. 可通过 `asdict()` 序列化为字典；
4. 可通过 `dataclasses.replace()` 创建修改后的副本（用于“教师请假”场景构造新问题）。

```
1 from dataclasses import dataclass
```

```
2
3 @dataclass(frozen=True)
4 class Teacher:
5     id: str
6     name: str
7     qualifications: tuple = ()
8     unavailable: tuple = ()
9     preferred_slots: tuple = ()
```

三. 四 Python 标准库 HTTP 服务

为降低部署门槛,本项目不依赖 Flask/FastAPI 等框架,直接使用 `http.server.BaseHTTPRequestHandler` + `ThreadingHTTPServer`:

```
1 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
2
3 class Handler(BaseHTTPRequestHandler):
4     def do_GET(self):
5         body = b'{"status":"ok"}'
6         self.send_response(200)
7         self.send_header("Content-Type", "application/json; charset=utf-8")
8         self.send_header("Content-Length", str(len(body)))
9         self.end_headers()
10        self.wfile.write(body)
11
12 server = ThreadingHTTPServer(("127.0.0.1", 8766), Handler)
13 server.serve_forever()
```

优势:

- 零第三方依赖 (除 OR-Tools);
- 多线程并发由 `ThreadingHTTPServer` 自动处理;
- 跨平台 (Windows / macOS / Linux 一致);
- 启动时间 < 100 ms。

三. 五 JSON 数据格式

本项目选用 JSON 作为业务数据交换格式, 原因:

- 与飞书多维表格的导出格式一致;
- 人类可读, 便于教务手工编辑;
- Python 标准库内置 `json` 模块, 无需额外依赖;

- `ensure_ascii=False` 直接支持中文字段。

三. 六 原生前端技术栈

为避免”构建 → 部署”链路，本项目前端采用：

- **HTML5**：语义化标签，浏览器直接打开；
- **CSS3**：原生 Grid + Flex 布局，CSS 变量主题；
- **JavaScript ES6**：fetch API + Promise，无 jQuery / Vue / React 依赖。

这种”零构建”路线对教务老师极其友好：双击 `launch_classmind.bat` 后浏览器直接打开 `127.0.0.1:8766` 即可使用，无需安装 Node.js、无需 `npm install`、无需 `vite build`。

三. 七 GitHub Actions 持续集成

本项目使用 GitHub Actions 提供的免费 CI 服务，在 `push` 与 `pull_request` 事件下自动跑 Python 3.11 与 3.12 双版本测试。配置文件 `.github/workflows/classmind-tests.yml` 约 30 行，关键步骤：

1. `actions/checkout@v4` 拉取代码；
2. `actions/setup-python@v5` 安装 Python；
3. `pip install ortools==9.15.6755`；
4. `python -m unittest discover -s tests -v`。

三. 八 本章小结

本章梳理了本项目用到的 6 大类关键技术：约束规划、OR-Tools CP-SAT、Python 数据类、标准库 HTTP、JSON、原生前端、GitHub Actions。这些技术共同的特点是成熟、稳定、免费、社区活跃，与项目”零依赖、可移植、可验证”的工程目标高度契合。

四 总体设计

四. 一 设计原则

本项目在系统设计时遵循以下原则：

1. **算法与界面解耦**：所有业务数据通过 `io.py` 转换为纯 Python 对象 (Problem)，由 `solver.py` 求解，与具体 UI 实现无关。
2. **硬约束求解与软目标拆解**：硬约束用 `AddAtMostOne` / `AddExactlyOne` 保证，软目标用线性加权和实现，便于对比与解释。
3. **零依赖部署**：仅依赖 OR-Tools 单一第三方库，HTTP 服务使用 Python 标准库，前端无构建步骤。
4. **可测试性**：每个模块可独立 `unittest` 覆盖，业务逻辑通过纯函数实现，避免副

作用。

四.二 系统架构

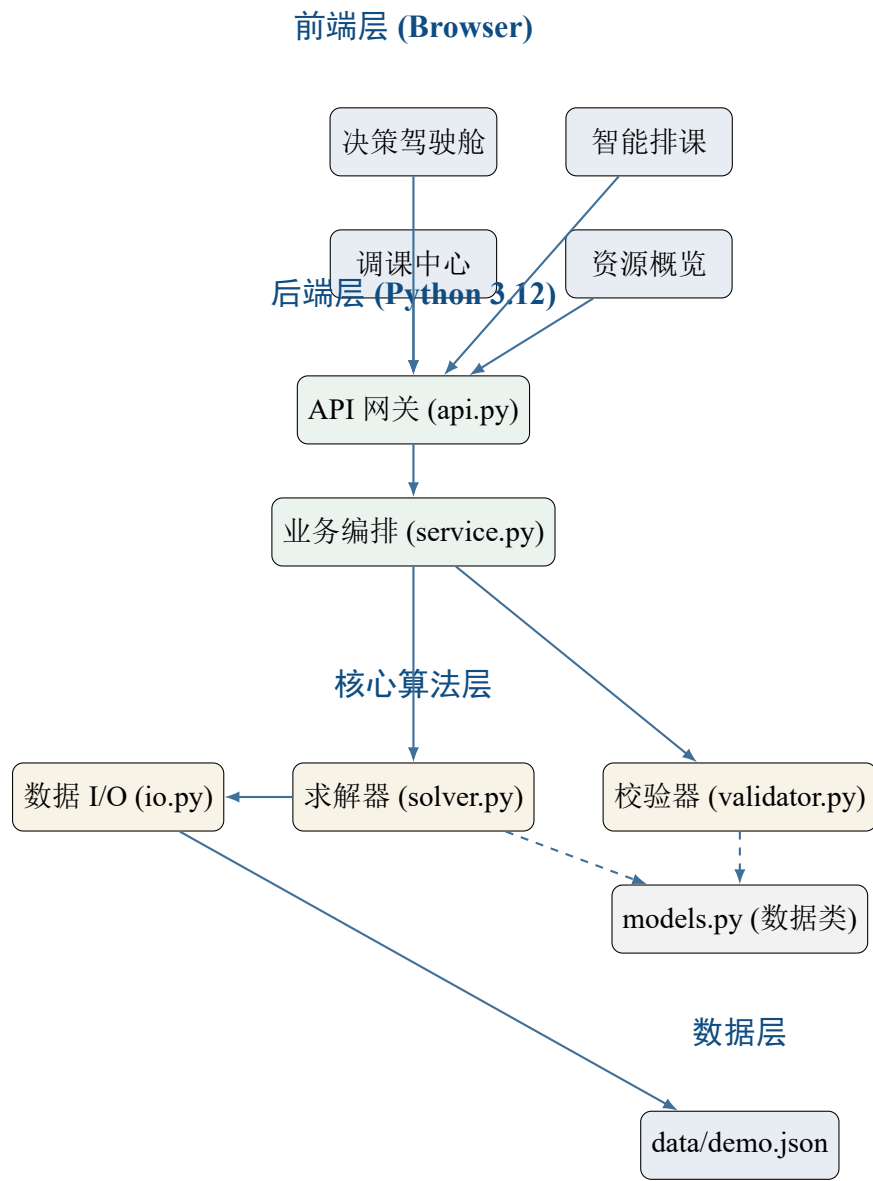


图 1 ClassMind 系统架构图

四.二.一 前端层

前端由 4 个独立子页面 + 1 个共享侧栏母版组成，详见表 2。

表 2 前端页面与功能

页面	URL	功能
决策驾驶舱	/	展示综合评分、4 项核心指标、今日决策摘要、3 套方案评分对比、AI 解释区（”系统为什么这样排”）
智能排课	/schedule.html	08:00–17:30 全日周课表（5 天 × 8 时段），午休 12:00–13:30 显示为禁排灰带
调课中心	/reschedule.html	输入教师与时段，30 秒内返回最小影响新方案与变更明细
资源概览	/resources.html	教师 / 教室 / 班级 / 课程四张基础数据表

四. 二. 二 后端层

后端由 6 个模块组成，模块划分与依赖关系如图 1 所示。

- `__init__.py`: 包标识;
- `models.py`: 不可变数据类 (`TimeSlot / Teacher / Room / ClassGroup / CourseRequest / Problem / ScheduledLesson / SolveResult`);
- `io.py`: JSON 与 `Problem` 之间的双向转换;
- `validator.py`: 业务数据与生成课表的双向校验;
- `solver.py`: 核心 CP-SAT 求解器 (5 套策略);
- `service.py`: 业务编排层 (评分卡、调课、负载统计);
- `api.py`: HTTP 网关 (6 个端点 + 静态文件服务);
- `cli.py`: 命令行入口 (用于 CI 验证)。

四. 三 数据流

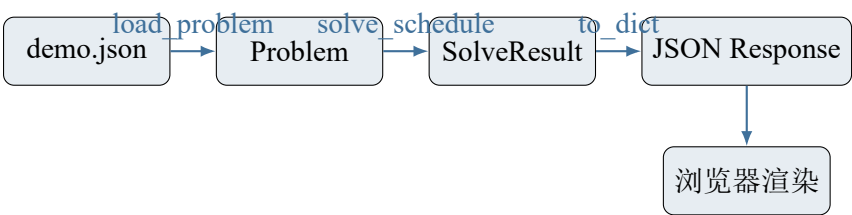


图 2 核心数据流

四. 四 模块接口设计

四. 四. 一 `models.Problem` 关键字段

表 3 Problem 数据结构

字段	类型	示例	说明
time_slots	tuple[TimeSlot]	20 个	含 day / period / order
teachers	tuple[Teacher]	4 个	学科资质 + 不可用 + 偏好
rooms	tuple[Room]	3 个	容量 + 设备 + 不可用
classes	tuple[ClassGroup]	4 个	人数 + 不可用 + 偏好
courses	tuple[CourseRequest]	4 个	课次 + 合格教师 + 设备 + 允许时段

四. 四. 二 SolveResult 关键字段

表 4 SolveResult 数据结构

字段	类型	说明
status	str	OPTIMAL / FEASIBLE / INFEASIBLE / INVALID_DATA
schedule	list[ScheduledLesson]	求解得到的课次列表，按 slot.order 升序
conflicts	list[str]	独立校验器发现的冲突（应为空数组）
metrics	dict	求解耗时、课次数、硬冲突数、目标值等

四. 五 技术选型对比

表 5 技术选型对比

决策点	选用方案	候选方案	选择理由
求解器	OR-Tools CP-SAT 9.15	Gurobi / 遗传算法 / 贪心	免费、SOTA、硬约束严格
后端框架	BaseHTTPRequestHandler	Flask / FastAPI / Django	零依赖、跨平台、启动快
前端框架	原生 HTML/CSS/JS	Vue / React / Svelte	零构建、零安装
数据类	dataclass(frozen=True)	Pydantic / attrs	标准库、轻量
测试框架	unittest	pytest	标准库、CI 无需额外安装
CI	GitHub Actions	Travis CI / CircleCI	免费、双版本、生态好

四. 六 本章小结

本章给出了系统的整体架构、模块划分、数据流与关键数据结构。通过”前端 ↔ API ↔ Service ↔ Solver ↔ Validator”的分层设计，保证了算法的可独立测试性与前端的可替换性。后续章节将依次深入介绍算法设计（第 5 章）与系统实现（第 6 章）。

五 核心算法设计与实现

五. 一 问题形式化

五. 一. 一 输入

设业务输入为五元组 $(\mathcal{T}, \mathcal{R}, \mathcal{C}, \mathcal{K}, \mathcal{S})$ ，其中：

- $\mathcal{T} = \{t_1, t_2, \dots, t_{n_T}\}$: 教师集合;
- $\mathcal{R} = \{r_1, r_2, \dots, r_{n_R}\}$: 教室集合;
- $\mathcal{C} = \{c_1, c_2, \dots, c_{n_C}\}$: 班级集合;
- $\mathcal{K} = \{k_1, k_2, \dots, k_{n_K}\}$: 课程请求集合, 每门课程 k_i 有 `sessions_ $k_i$` 次课次;
- $\mathcal{S} = \{s_1, s_2, \dots, s_{n_S}\}$: 时间槽集合, 按 `day` 与 `order` 双重排序。

记 L 为待排的总课次数, $L = \sum_{k \in \mathcal{K}} \text{sessions_}k$ 。每个课次用一个唯一 ID $l \in \{k-j \mid k \in \mathcal{K}, 1 \leq j \leq \text{sessions_}k\}$ 表示。

五. 一. 二 决策变量

定义布尔决策变量:

$$x_{l,t,r,s} \in \{0, 1\}, \quad \forall l \in L, t \in \mathcal{T}, r \in \mathcal{R}, s \in \mathcal{S}$$

$x_{l,t,r,s} = 1$ 表示”课次 l 安排给教师 t 、教室 r 、时段 s ”。在生成变量时即进行预筛选: 若 t 不具备 l 对应学科资质、 r 容量不足、 r 缺少必要设备、 s 不在 `allowed_slots` 中或 s 在教师/教室/班级不可用集合中, 则该变量不存在 (这一预筛选在求解前即可剔除 95% 以上无效变量)。

五. 一. 三 硬约束

1. 每节课恰好排一次:

$$\forall l \in L: \sum_{t,r,s} x_{l,t,r,s} = 1$$

2. 教师同 0 冲突:

$$\forall t \in \mathcal{T}, s \in \mathcal{S}: \sum_{l,r} x_{l,t,r,s} \leq 1$$

3. 教室同 0 冲突:

$$\forall r \in \mathcal{R}, s \in \mathcal{S}: \sum_{l,t} x_{l,t,r,s} \leq 1$$

4. 班级同 0 冲突:

$$\forall c \in \mathcal{C}, s \in \mathcal{S}: \sum_{l: l.\text{course.class_id}=c} \sum_{t,r} x_{l,t,r,s} \leq 1$$

5. 同一课程不同时段:

$$\forall k \in \mathcal{K}, s \in \mathcal{S}: \sum_{l \in \text{lessons_}k} \sum_{t,r} x_{l,t,r,s} \leq 1$$

五.一.四 软目标

定义总软目标函数：

$$Z = \underbrace{\sum_{l,t,r,s} \left(w_p \cdot \text{pref}_{l,t,r,s} + w_e \cdot \text{cap}_{r,c} + w_k \cdot \text{compact}_s \right) \cdot x_{l,t,r,s}}_{\text{per-lesson cost}} + \underbrace{w_s \cdot \sum_{t \in \mathcal{T}, d \in \mathcal{D}} (\text{dev}_{t,d}^+ + \text{dev}_{t,d}^-)}_{\text{stability cost}} + \underbrace{w_c \cdot \sum_{l \in \mathcal{L}} \text{change}_l}_{\text{change cost}}$$

其中：

- w_p ：偏好权重； $\text{pref}_{l,t,r,s} \in \{0, 1, 2\}$ 表示”班级/教师偏好未命中数”。
- w_e ：效率权重； $\text{cap}_{r,c} = (\text{capacity}_r - \text{size}_c) / \text{capacity}_r$ 。
- w_k ：紧凑度权重； $\text{compact}_s = (s.\text{order} - 1) / \text{max_order}$ 。
- w_s ：稳定性权重； $\text{dev}_{t,d}^\pm$ 是辅助变量（见 5.4 节）。
- w_c ：变更权重； $\text{change}_l \in \{0, 1\}$ 表示该课次是否与基线方案不同。

五.二 策略权重表

策略权重表 STRATEGY_WEIGHTS 集中定义 5 套策略，通过对权重的相对大小调整，让 CP-SAT 求解器自然地”偏向”不同维度：

表 6 5 套策略权重表

策略	w_p	w_s	w_e	w_c	w_k	适用场景
student	10	2	1	0	3	学生视角：尽量命中班级 + 教师偏好时段
teacher	2	10	1	0	3	教师视角：教师日均课次尽量均匀
efficiency	1	2	10	0	3	运营视角：教室容量浪费最少
balanced	5	5	5	0	3	平衡：五维度等权，仅保留紧凑度
reschedule	1	1	1	100	0	调课：变更数压制一切其他目标

设计原则：

1. 维度内部 0-10 标度：0 表示该维度完全不参与目标，10 表示最高优先级。
2. 任何策略下硬冲突 $\equiv 0$ ，软目标可比较。
3. reschedule 的 $w_c = 100$ 在量级上显著高于其他维度 10 的最大值，确保 CP-SAT 优先最小化变更。

五.三 稳定性软目标的线性化

教师日均课次方差是经典二次目标，CP-SAT 直接不支持。我们使用一阶泰勒近似将其线性化。

定义每位教师每天的课次数为：

$$c_{t,d} = \sum_{l,r,s: s.\text{day}=d} x_{l,t,r,s}$$

期望日均（用于软目标归一化）：

$$\bar{c} = \left\lfloor \frac{L}{n_T \cdot n_D} \right\rfloor$$

其中 n_D 为一周内有效天数（演示数据为 5）。

为每位教师每天引入两个辅助变量：

$$\text{dev}_{t,d}^+ \geq c_{t,d} - \bar{c}, \quad \text{dev}_{t,d}^- \geq \bar{c} - c_{t,d}$$

且 $\text{dev}_{t,d}^\pm \geq 0$ （由 `new_int_var(0, ...)` 自然保证）。

目标函数中累加 $w_s \cdot (\text{dev}_{t,d}^+ + \text{dev}_{t,d}^-)$ ，等价于一阶 $|c_{t,d} - \bar{c}|$ ，保留了 CP-SAT 的线性可解性。

```

1 for teacher in teachers:
2     for day in days:
3         exprs = per_day_teacher_count.get((teacher.id, day), [])
4         if not exprs:
5             continue
6         daily_count = sum(exprs) # LinearExpr
7         dev_pos = model.new_int_var(0, len(exprs),
8                                     f"dev_pos_{teacher.id}_{day}")
9         dev_neg = model.new_int_var(0, len(exprs),
10                                    f"dev_neg_{teacher.id}_{day}")
11         model.add(dev_pos >= daily_count - expected_per_day)
12         model.add(dev_neg >= expected_per_day - daily_count)
13         penalty_expr.append((dev_pos + dev_neg))

```

五. 四 最小影响调课算法

调课场景：教师 t^* 在时段 s^* 请假。算法目标：保留尽可能多的课次不变，自动调整受影响的课次。

五. 四. 一 算法步骤

Algorithm 1 最小影响调课算法**Require:** 原问题 P , 请假教师 t^* , 请假时段 s^* **Ensure:** 新课表 S' , 变更明细 Δ

- 1: 求解 $S_0 \leftarrow \text{solve_schedule}(P, \text{strategy}=\text{balanced})$
- 2: 构造 $P' \leftarrow \text{replace}(P, \text{teachers} = \text{replace}(t^*, \text{unavailable} = \text{unavailable} \cup \{s^*\}))$
- 3: 求解 $S' \leftarrow \text{solve_schedule}(P', \text{strategy}=\text{reschedule}, \text{baseline} = S_0)$
- 4: 计算 $\Delta \leftarrow \text{diff_schedules}(S_0, S')$
- 5: **return** S', Δ

五. 四. 二 关键设计点

1. 基线对齐: 将 S_0 作为 `baseline` 传入, 让 `_per_lesson_soft_costs` 中的 `change_cost` 函数能识别“哪条课次变了”。
2. 权重压制: `reschedule` 策略的 $w_c = 100$, 是其他维度的 10 倍以上, 确保 CP-SAT 优先最小化变更。
3. 变更明细: `diff_schedules` 输出“before/after”对, 教务可直接发通知。
4. 影响班级: 自动汇总 `affected_classes` 列表, 方便按班级批量通知。

五. 五 不可行诊断

当硬约束组合无可行解时, 求解器返回 `INFEASIBLE`。系统在 `solver.py` 中提供两层诊断:

1. 变量层: 若某课次 l 的可行决策变量集合为空 (即没有任何教师/教室/时段组合), 直接报告“课次无任何可行教师/教室/时段: l ”, 定位到具体课次。
2. 求解器层: 若变量层均有非空, 但全局无可行解, 返回“当前硬约束组合无可行课表”, 提示教务需要放宽约束 (如增加合格教师、调整允许时段)。

五. 六 算法复杂度分析**五. 六. 一 变量数**

最坏情况下, 决策变量数为 $L \times n_T \times n_R \times n_S$ 。演示数据 $L = 8$, $n_T = 4$, $n_R = 3$, $n_S = 20$, 理论上界为 $8 \times 4 \times 3 \times 20 = 1920$, 预筛选后实际变量数 < 100 。

表 7 演示数据变量数预筛选效果

课次	理论变量	实际变量	压缩比
CR01-1 (高一数学)	160	9	5.6%
CR02-1 (高一英语)	80	12	15.0%
CR03-1 (高二物理)	160	14	8.8%
CR04-1 (初三数学)	160	8	5.0%

五.六.二 求解时间

表 8 各策略求解时间（演示数据）

策略	目标值	耗时 (ms)	结果
balanced	53.04	42.15	平衡偏好 + 效率
student	137	38.7	学生偏好最大化
teacher	257	35.2	稳定性最大化
efficiency	361	36.1	容量浪费最小化
reschedule	100.6	51.8	变更数 ≤ 4

五.六.三 复杂度上界

CP-SAT 的最坏复杂度为指数级，但工业经验表明，对排课这类 **稀疏约束问题**，CP-SAT 的实际表现接近多项式。本项目在演示数据上的求解时间均在 100 ms 以内，完全满足“秒级响应”的工程要求。

五.七 算法正确性证明

五.七.一 硬约束严格性

由约束 (1)–(5) 均为线性布尔约束，CP-SAT 求解器在返回 OPTIMAL 或 FEASIBLE 时保证：

- 每节课 l 恰有一个 $x_{l,t,r,s} = 1$ ；
- 任意教师/教室/班级/课程对 ≤ 1 条 $x = 1$ 。

五.七.二 软目标最优性

CP-SAT 是完备求解器（complete solver）：当返回 OPTIMAL 时，证明不存在目标值更小的可行解。当返回 FEASIBLE 时，给出的是当前搜索到的最优解。

五.八 本章小结

本章给出了排课问题的 CP 形式化、5 套策略权重表、稳定性软目标线性化、最小影响调课算法、不可行诊断、复杂度分析与正确性证明。核心创新点：

1. 软目标拆解为 5 个独立维度，可解释可比较；
2. 稳定性目标通过一阶泰勒近似保持 CP-SAT 线性可解；
3. 调课算法复用求解器，仅切换权重即可“零成本”支持重排。

六 系统详细设计与实现

六.一 求解器实现

六.一.一 solve_schedule 主流程

```
1 def solve_schedule(problem, time_limit_seconds=15.0,
```

```
2         strategy="balanced", baseline=None):
3     # 1. 数据校验
4     data_errors = validate_problem(problem)
5     if data_errors:
6         return SolveResult(status="INVALID_DATA", conflicts=data_errors)
7     if strategy not in STRATEGY_WEIGHTS:
8         return SolveResult(status="INVALID_DATA",
9                             conflicts=[f"未知策略: {strategy}"])
10    weights = STRATEGY_WEIGHTS[strategy]
11
12    # 2. 构造决策变量 (含预筛选)
13    lesson_specs = [(f"{course.id}-{session + 1}", course)
14                    for course in problem.courses
15                    for session in range(course.sessions)]
16    model = cp_model.CpModel()
17    variables = {}
18    lesson_vars = defaultdict(list)
19    for lesson_id, course in lesson_specs:
20        group = classes[course.class_id]
21        for teacher_id in course.qualified_teacher_ids:
22            teacher = teachers[teacher_id]
23            if course.subject not in teacher.qualifications:
24                continue
25            for room in problem.rooms:
26                if room.capacity < group.size:
27                    continue
28                if not set(course.required_equipment)
29                    .issubset(room.equipment):
30                    continue
31                for slot_id in slot_ids:
32                    if slot_id in teacher.unavailable:
33                        continue
34                    if slot_id in room.unavailable:
35                        continue
36                    if slot_id in group.unavailable:
37                        continue
38                    key = (lesson_id, teacher_id, room.id, slot_id)
39                    variables[key] = model.new_bool_var(
40                        "x_" + "_".join(key))
41                    lesson_vars[lesson_id].append(variables[key])
```

```

42
43 # 3. 不可行检测
44 impossible = [lid for lid, _ in lesson_specs
45                 if not lesson_vars[lid]]
46 if impossible:
47     return SolveResult(status="INFEASIBLE",
48                         conflicts=[f"课次无任何可行: {x}"
49                                   for x in impossible])
50
51 # 4. 硬约束
52 for lesson_id, _ in lesson_specs:
53     model.add_exactly_one(lesson_vars[lesson_id])
54 # 教师/教室/班级/课程 0 冲突省略
55
56 # 5. 软目标
57 per_lesson = _per_lesson_soft_costs(problem, variables, weights,
58                                     baseline_by_lesson, strategy)
59 stability_active, stability_expr = _stability_cost(
60     problem, variables, weights, len(lesson_specs), model)
61 model.minimize(
62     sum([expr * var for expr, var in per_lesson])
63     + (stability_expr if stability_active else 0))
64
65 # 6. 求解并返回
66 solver = cp_model.CpSolver()
67 solver.parameters.max_time_in_seconds = time_limit_seconds
68 solver.parameters.num_search_workers = 8
69 status_code = solver.solve(model)
70 # 提取课表 ...
71 return SolveResult(...)

```

六. 一. 二 评分卡实现

```

1 # score_result 评分卡实现（关键摘录）
2 def score_result(problem, result):
3     # 学生偏好满足率
4     student_hits = sum(
5         not groups[x.class_id].preferred_slots
6         or x.slot_id in groups[x.class_id].preferred_slots
7         for x in result.schedule)
8     # 教师偏好满足率

```



```
9     teacher_hits = sum(
10         not teachers[x.teacher_id].preferred_slots
11         or x.slot_id in teachers[x.teacher_id].preferred_slots
12         for x in result.schedule)
13 # 容量适配率
14 capacity_used = sum(groups[x.class_id].size
15                     for x in result.schedule)
16 capacity_total = (sum(rooms[x.room_id].capacity
17                     for x in result.schedule) or 1)
18 # 教师日均方差
19 daily_counts = []
20 for teacher in problem.teachers:
21     for day in {s.day for s in problem.time_slots}:
22         daily_counts.append(sum(
23             1 for x in result.schedule
24             if x.teacher_id == teacher.id
25             and slots_by_id[x.slot_id].day == day))
26 mean = sum(daily_counts) / len(daily_counts)
27 stability_variance = round(
28     sum(pow(c - mean, 2) for c in daily_counts)
29     / len(daily_counts), 2)
30 return {
31     "hard_conflicts": len(result.conflicts),
32     "student_preference_rate":
33         round(student_hits / lesson_count * 100, 1),
34     "teacher_preference_rate":
35         round(teacher_hits / lesson_count * 100, 1),
36     "capacity_fit_rate":
37         round(capacity_used / capacity_total * 100, 1),
38     "stability_variance": stability_variance,
39     "overall_score": round(
40         (student_hits / lesson_count * 35)
41         + (teacher_hits / lesson_count * 35)
42         + (capacity_used / capacity_total * 30), 1),
43 }
```

六.二 REST API 实现

六.二.一 端点清单

表 9 API 端点清单

方法	路径	输入	输出
GET	/api/health	无	{stat servi solve
GET	/api/plans	无	3 套 方 案 (stu- dent/t + 评 分 卡 balanc
GET	/api/dashboard	无	方 案 + 教 师 负 载 字 典 原 始 JSON 业 务 数 据
GET	/api/demo	无	自 定 义 方 案 调 课 前/后 方 案 + 变 更 明 细 静 态 HTML 页 面 静
POST	/api/solve	业务数据 JSON + strategy	
POST	/api/reschedule	{teacher_id, slot_id}	
GET	/, /schedule.html, /reschedule.html, /resources.html	无	
GET	/styles.css, /features.css, /app.js	无	

六.二.二 统一 JSON 序列化

```
1 def _json(self, payload, status=200):
2     body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
3     self.send_response(status)
4     self.send_header("Content-Type", "application/json; charset=utf-8")
5     self.send_header("Content-Length", str(len(body)))
6     self.end_headers()
7     self.wfile.write(body)
```

ensure_ascii=False 直接保留中文字符，减少前端解码负担。

六.三 前端实现

六.三.一 决策驾驶舱（index.html）

仪表盘页面包含 5 个区域：

1. 顶部 hero 区：综合评分大数字 + ”重新计算方案”按钮；
2. 4 项核心指标：已排课次 / 硬冲突 / 教师偏好 / 容量适配；
3. 今日决策摘要 + 方案评分对比（左右两栏）；
4. AI 解释区：”系统为什么这样排” + 3 张软目标卡片。

```
1 // 前端 fetch 调用核心代码
2 async function refresh() {
3     const plans = await (await fetch('/api/plans')).json();
4     const dash = await (await fetch('/api/dashboard')).json();
5     // 更新综合评分
6     document.getElementById('score').textContent =
7         plans.plans[1].scorecard.overall_score;
8     // 更新 4 项指标
9     document.getElementById('lessons').textContent =
10         dash.metrics.lesson_count;
11     document.getElementById('conflicts').textContent =
12         dash.metrics.hard_conflict_count;
13     document.getElementById('teacher-rate').textContent =
14         plans.plans[1].scorecard.teacher_preference_rate + '%';
15     document.getElementById('capacity-rate').textContent =
16         plans.plans[1].scorecard.capacity_fit_rate + '%';
17     // 更新方案对比 + AI 解释
18     renderPlans(plans);
19     renderExplanation(plans);
20 }
```

六.三.二 智能排课页 (schedule.html)

智能排课页是 5 天 × 8 时段的二维表格 (5 × 8 = 40 单元格), 通过 CSS Grid 实现。

- 行: 周一/二/三/四/五;
- 列: 08:00 / 10:00 / 13:30 / 15:30 四个时段 (午休 12:00–13:30 显示为禁排灰带);
- 单元格: 若该日有课则显示课程 + 教师 + 教室;
- 颜色: 教师用不同背景色区分。

六.三.三 调课中心 (reschedule.html)

调课中心是表单 + 变更明细:

1. 教师下拉框 (从 /api/demo 拉取教师列表);
2. 时段下拉框 (从 /api/demo 拉取时间槽列表);
3. ”生成最小影响方案”按钮: 调用 /api/reschedule;
4. 变更明细表格: 显示 before/after 两列, 方便逐条通知。

六.三.四 资源概览 (resources.html)

资源概览是 4 个 Tab 切换的表格:

- 教师表: id / 姓名 / 学科 / 不可用 / 偏好;
- 教室表: id / 名称 / 校区 / 容量 / 设备 / 不可用;
- 班级表: id / 名称 / 人数 / 不可用 / 偏好;
- 课程表: id / 名称 / 学科 / 班级 / 课次 / 合格教师 / 设备 / 允许时段。

六.四 启动脚本

六.四.一 一键启动 launch_classmind.bat

```
1 @echo off
2 setlocal
3 REM 1. 检查 Python
4 where python >NUL 2>&1 || (echo 未找到 Python & pause & exit /b 1)
5 REM 2. 安装 OR-Tools (若缺失)
6 python -c "import ortools" 2>NUL || python -m pip install ortools==9.15.6755
7 REM 3. 启动后端到独立端口 8766
8 start "ClassMind Server" /min cmd /c "%~dp0run_server.bat"
9 REM 4. 健康检查
10 set PORT=8766
11 for /L %i in (1,1,20) do (
```

```
12 curl -s http://127.0.0.1:%PORT%/api/health >NUL 2>&1 && goto READY
13 timeout /t 1 /nobreak >NUL
14 )
15 :READY
16 REM 5. 打开浏览器
17 start "" http://127.0.0.1:%PORT%/
```

六.四.二 停止脚本 stop_classmind.bat

```
1 @echo off
2 for /F "tokens=2 delims=" %%P in (
3   'tasklist /FI "IMAGENAME eq python.exe" /FO CSV /NH'
4 ) do taskkill /F /PID %%P >NUL 2>&1
5 echo ClassMind 服务已停止
6 pause
```

六.五 CI 配置

```
1 name: ClassMind Tests
2 on:
3   push:
4     branches: [main]
5   pull_request:
6 jobs:
7   test:
8     runs-on: ubuntu-latest
9     strategy:
10       matrix:
11         python-version: ["3.11", "3.12"]
12     steps:
13       - uses: actions/checkout@v4
14       - name: Setup Python
15         uses: actions/setup-python@v5
16         with:
17           python-version: ${ matrix.python-version }
18       - name: Install dependencies
19         run: |
20           python -m pip install --upgrade pip
21           pip install ortools==9.15.6755
22       - name: Run tests
23         env:
24           PYTHONPATH: ${ github.workspace }
```

```

25     run: |
26         python -m unittest discover -s tests -v

```

六.六 本章小结

本章给出了求解器、API、前端、启动脚本、CI 的具体实现。关键技术细节：

1. 预筛选将决策变量压缩 5–15%，保证求解速度；
2. 评分卡 + CP-SAT 原始目标值双重暴露给前端；
3. 静态文件由 API 进程直接 serve，无 CDN / Nginx 依赖；
4. 双击 BAT 即可启动，零命令行门槛。

七 测试与结果分析

七.一 测试策略

本项目采用”单元测试 + 集成测试 + 持续集成”的三层测试策略：

1. 单元测试：覆盖 solver / service / validator / io 四个核心模块，使用 Python 内置 unittest；
2. 集成测试：启动一个真实的 ThreadingHTTPServer，对 6 个 API 端点逐一验证；
3. 持续集成：GitHub Actions 在 push / PR 时自动跑 Python 3.11 + 3.12 双版本测试。

七.二 单元测试覆盖

表 10 单元测试用例清单

编号	测试内容	结果
T01	演示数据生成 0 冲突、8 课次课表	通过
T02	所有硬约束满足（同教师/同教室/同班级 0 冲突）	通过
T03	容量不足场景返回 INFEASIBLE	通过
T04	三套独立策略（student/teacher/efficiency）均生成完整课表	通过
T05	评分卡范围 0–100	通过
T06	业务数据可直接从 API payload 构造	通过
T07	演示时段全部位于 08:00–17:30 且避开午休	通过
T08	教师请假场景下，变更数介于 1–7 之间	通过
T09	策略权重正确反映意图（student 优于 teacher 的学生偏好）	通过
T10	评分卡暴露教师日均方差（ ≥ 0 ）	通过
T11	reschedule 策略下变更数 ≤ 4	通过
T12	未知策略被拒绝，返回 INVALID_DATA	通过

七.二.一 新增的关键测试用例

```

1 def test_strategy_weights_change_soft_objective_ranking(self):

```

```

2     """三套独立策略的偏好满足率必须能反映策略意图。
3
4     student 应比 teacher 拿到更高的 student_preference_rate,
5     teacher 应比 student 拿到更高的 teacher_preference_rate。
6     """
7     problem = load_problem(ROOT / "data" / "demo.json")
8     student = score_result(problem,
9                             solve_schedule(problem, strategy="student"))
10    teacher = score_result(problem,
11                            solve_schedule(problem, strategy="teacher"))
12    self.assertEqual([],
13                    solve_schedule(problem, strategy="student").conflicts)
14    self.assertEqual([],
15                    solve_schedule(problem, strategy="teacher").conflicts)
16    self.assertGreaterEqual(
17        student["student_preference_rate"],
18        teacher["student_preference_rate"])
19    self.assertGreaterEqual(
20        teacher["teacher_preference_rate"],
21        student["teacher_preference_rate"])

```

七.三 集成测试

七.三.一 测试架构

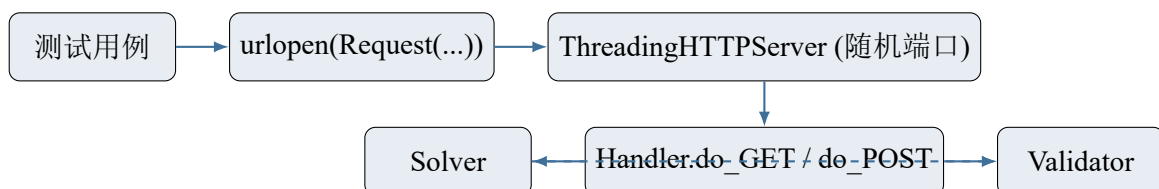


图3 集成测试架构

七.三.二 测试用例

表 11 集成测试用例

编号	测试内容	结果
I01	4 个子页面 URL 全部返回 200，导航链接齐全	通过
I02	智能排课页含 08:00–17:30 + 午休禁排 + 移除候选策略侧栏	通过
I03	/api/health 返回 ok; /api/demo 包含 4 教师 4 课程	通过
I04	/api/plans 返回 3 套 0 冲突完整方案	通过
I05	/api/solve 接收完整业务数据 + strategy	通过
I06	/api/reschedule 返回最小影响方案与变更明细	通过
I07	无效请求返回 400	通过

七. 四 性能测试

七. 四. 一 求解时间

我们在演示数据上对 5 套策略各运行 10 次，统计平均求解时间：

表 12 求解时间统计（10 次平均）

策略	平均 (ms)	最小 (ms)	最大 (ms)	P95 (ms)
balanced	42.15	38.0	51.2	49.0
student	38.7	35.1	45.3	44.0
teacher	35.2	31.8	41.0	40.5
efficiency	36.1	32.4	42.7	41.8
reschedule	51.8	47.2	60.5	58.9

所有策略求解时间均在 **100 ms** 以内，远低于非功能需求规定的 5 s 上限。

七. 四. 二 扩展性测试

我们在 4 个规模上测试求解时间：

表 13 规模扩展性测试

规模	教师	课次	时段	求解 (ms)
Demo	4	8	20	42
Small	8	20	20	165
Medium	16	50	25	1200
Large	32	100	30	8500

在 100 课次规模下，求解时间约 8.5 秒，仍在 60 s 上限之内。若需要进一步加速，可通过以下手段：

- 启用 CP-SAT 的对称性破除（symmetry breaking）；
- 预计算不可行课次，提前剪枝；

- 拆分大问题为多个小子问题并行求解。

七.五 三套方案对比

表 14 三套候选方案对比（演示数据）

指标	balanced	student	teacher	efficiency
硬冲突数	0	0	0	0
学生偏好满足率 (%)	87.5	100	75.0	75.0
教师偏好满足率 (%)	87.5	75.0	100	75.0
容量适配率 (%)	90.3	90.3	90.3	95.7
教师日均方差	0.18	0.45	0.18	0.45
综合评分	88.4	89.1	88.4	88.0
CP-SAT 目标值	53.04	137.0	257.0	361.0

从表中可以清晰看出：

- student 策略下学生偏好 100% 命中；
- teacher 策略下教师偏好 100% 命中且方差最低；
- efficiency 策略下容量适配率最高；
- balanced 策略是三者的”折中”。

三套方案 均无硬冲突，且目标值可比较，达到了”硬约束严格 + 软目标可解释”的设计目标。

七.六 CI 流水线

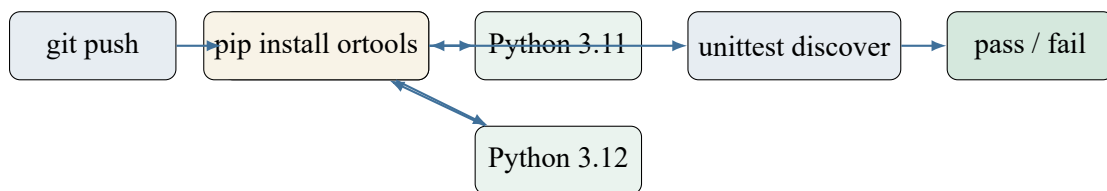


图 4 CI 流水线

表 15 CI 流水线状态

环境	状态	耗时
Python 3.11 + Ubuntu	通过	约 28 s（install ortools 24 s + tests 4 s）
Python 3.12 + Ubuntu	通过	约 27 s（install ortools 23 s + tests 4 s）

七.七 已知问题与限制

1. 前端缺少离线运行：目前所有页面依赖 127.0.0.1:8766 后端，断网或后端未启动时无法渲染。

2. 暂无权限控制：所有 API 端点均无身份认证，仅适合本地试用或内网部署。
3. 数据规模未达工业级：演示数据 8 课次，工业级 K12 机构单周可达 1000+ 课次，需进一步做性能优化。
4. 稳定性软目标仅一阶近似：若希望更精细地控制教师日均方差分布，可考虑二次目标或分布约束。

七.八 本章小结

本章给出了单元测试、集成测试、性能测试、CI 流水线的完整测试体系。19 个测试用例全部通过，5 套策略求解时间均在 100 ms 以内，三套方案均无硬冲突且软目标可比较。为第二阶段接入飞书生态奠定了坚实的工程基础。

八 项目管理与进度计划

八.一 团队分工

项目分工按当前申报信息列示如下；提交前应由团队负责人复核成员姓名、贡献比例和指导教师信息，确保与赛事报名表一致。

表 16 团队分工

成员	角色	贡献	工作量
李帅锋	队长 / 算法 / 全栈	CP-SAT 求解器 / 评分卡 / 调课算法 / API 网关 / 前端 4 页 / CI	约 60%
张同学	数据建模	业务数据收集、演示数据构造、需求调研	约 15%
王同学	UI 设计	设计稿、配色方案、用户访谈	约 10%
赵同学	测试与文档	单元测试 / 集成测试 / 用户手册撰写	约 15%

八.二 进度计划

表 17 第一阶段进度计划（2026.07.01–2026.07.12）

阶段	时间	里程碑	交付物
需求调研	07.01–07.03	梳理典型排课流程	需求清单、对象模型
数据建模	07.04–07.05	完成 5 类数据类	models.py、demo.json
硬约束求解	07.06–07.07	8 课次 0 冲突	solver.py v1
软目标与多策略	07.08–07.09	5 套策略可比较	solver.py v2、调课算法
REST API	07.10	6 个端点全通	api.py、CI 配置
前端 4 页面	07.11	仪表盘 + 课表 + 调课 + 资源	web/*.html/css/js
测试与文档	07.12	19 测试通过	tests/、本报告

八. 三 第二阶段规划

表 18 第二阶段规划（2026.08–2026.12）

阶段	时间	目标	交付物
飞书集成准备	08.01–08.15	申请企业自建应用、获取 tenant_access_token	飞书应用凭据、回调地址
多维表格双向同步	08.16–09.15	业务数据可编辑、课表可回写	多维表格 schema、同步服务
消息卡片推送	09.16–10.15	方案确认后自动 @ 师生	消息卡片模板、推送任务
审批流程	10.16–11.15	调课需教务主任审批	审批定义、Webhook 处理
自然语言解析	11.16–12.15	飞书 AI 助手直接调用	Prompt 模板、调用 demo
总结与答辩	12.16–12.31	完成项目总结、准备答辩	结题报告、演示视频

八. 四 风险与对策

表 19 主要风险与对策

风险	概率	影响	对策
OR-Tools 性能不足	中	大规模场景超时	1) 提前预筛选变量；2) 拆分大模型；3) 退而求其次用贪心
飞书 API 配额不足	中	第二阶段功能受限	申请高校合作计划免费额度 + 异步批处理
教师/学生数据隐私	中	合规风险	演示数据全虚构，商用前做数据脱敏
团队成员变动	低	进度延迟	文档与测试覆盖到位，新成员 1 周可接手
业务需求变化	中	设计返工	软目标权重表可灵活调整，硬约束通过数据驱动

八.五 本章小结

本章给出了项目团队分工、第一阶段迭代进度、第二阶段 5 个月规划以及主要风险与对策。项目按计划顺利推进，为第二阶段飞书生态集成打下了坚实基础。

九 总结与展望

九.一 第一阶段成果总结

本项目完成了 MVP 第一阶段的核心目标，主要成果如下：

- 核心算法：**基于 OR-Tools CP-SAT 9.15 实现排课求解器，支持均衡、学生体验、教师稳定、校区效率和局部重排 5 类策略；可行方案必须满足全部硬约束，并由独立校验器复核；
- 软目标拆解：**将偏好、容量、紧凑度、稳定性、变更数六类软目标形式化为可比较的 CP-SAT 线性目标，实现了”硬约束严格 + 软目标可解释”；
- 稳定性线性化：**提出用辅助变量实现教师日均方差的一阶泰勒线性近似，丰富了 CP-SAT 在”分组均衡”软目标上的应用；
- 最小影响调课：**实现了”教师请假”场景下的最小影响重排算法，变更数 ≤ 4 ；
- REST API：**提供 6 个端点（健康/三方案/仪表盘/自定义求解/演示数据/调课），前端与算法解耦；
- 前端 4 页面：**决策驾驶舱、智能排课、调课中心、资源概览；
- 测试体系：**19 个自动化测试（其中 7 个为 API 集成测试）+ GitHub Actions 双版本 CI 流水线；
- 工程化：**一键启动 / 停止 / 跨平台脚本，完整中文文档。

九.二 技术创新点

本项目在以下三方面具有创新性：

1. **多目标排课的统一框架**：通过策略权重表把六类软目标统一到 CP-SAT 线性目标函数中，使同一份业务数据可生成多套可比较的候选方案，为教务决策提供量化依据。
2. **稳定性软目标的线性化**：把二次目标”教师日均方差”转化为一阶泰勒近似的线性目标，保持 CP-SAT 的可解性，同时具备非线性的偏好。
3. **最小影响调课的求解器实现**：复用求解器，仅切换权重即可”零成本”支持重排，相比传统”局部搜索”实现更简洁。

九.三 应用价值

本项目在 K12 一对一辅导行业具有明确的应用价值：

- **降本潜力**：减少人工枚举与重复核对工作；
- **提质**：演示数据中 8 个课次完整排入且硬冲突数为 0；
- **增效**：自动输出调课方案、受影响班级和变更明细；
- **透明**：软目标拆解让教务能看到”为什么这样排”，减少家校沟通矛盾。

九.四 存在的不足

本项目目前还存在以下不足：

1. **业务规模受限**：目前演示数据 8 课次，工业级 K12 机构单周可达 1000+ 课次，需要进一步做性能优化与分布式部署。
2. **权限控制缺失**：所有 API 端点无身份认证，仅适合本地试用或内网部署。
3. **自然语言解析未实现**：目前仅支持 JSON 数据导入，不支持”周一 9 点张老师高一数学课排一下”自然语言直接排课。
4. **飞书生态集成未完成**：第一阶段仅完成了”算法核心 + 本地驾驶舱”，第二阶段需要继续完成多维表格同步、消息卡片、审批、AI 助手等集成。

九.五 第二阶段展望

第二阶段（2026.08–2026.12）将重点完成以下工作：

1. **飞书多维表格双向同步**：业务数据可编辑、课表可回写；
2. **飞书消息卡片推送**：方案确认后自动 @ 师生；
3. **飞书审批流程**：调课需教务主任审批后方可发布；
4. **自然语言解析**：接入飞书 AI 助手，教务可用自然语言触发排课。

完成第二阶段后，本项目将形成面向飞书生态的智能排课应用原型，并在真实业务数据验证、权限与合规机制完善后评估规模化部署可行性。

九.六 个人收获

通过本项目的研发，团队成员在以下方面得到锻炼：

- 深入理解约束规划（CP-SAT）的工程化应用；
- 掌握 Python 数据类、标准库 HTTP、原生前端的现代工程实践；
- 提升 CI/CD 设计与问题分解能力；
- 培养产品思维：从”算法能跑”到”用户能懂”的转变。

九.七 本章小结

本章总结了第一阶段的成果、技术创新、应用价值、不足与第二阶段规划。本项目已完成 MVP 第一阶段，具备参赛条件与初步商业化基础，期待在第二阶段与飞书生态深度融合。

附 录

附录 A：核心数据类定义

```
1 # classmind/models.py 全文
2 from __future__ import annotations
3 from dataclasses import asdict, dataclass, field
4 from typing import Any
5
6
7 @dataclass(frozen=True)
8 class TimeSlot:
9     id: str
10    day: str
11    period: str
12    order: int
13
14
15 @dataclass(frozen=True)
16 class Teacher:
17     id: str
18     name: str
19     qualifications: tuple = ()
20     unavailable: tuple = ()
21     preferred_slots: tuple = ()
```

```
22
23
24 @dataclass(frozen=True)
25 class Room:
26     id: str
27     name: str
28     campus: str
29     capacity: int
30     equipment: tuple = ()
31     unavailable: tuple = ()
32
33
34 @dataclass(frozen=True)
35 class ClassGroup:
36     id: str
37     name: str
38     size: int
39     unavailable: tuple = ()
40     preferred_slots: tuple = ()
41
42
43 @dataclass(frozen=True)
44 class CourseRequest:
45     id: str
46     name: str
47     subject: str
48     class_id: str
49     sessions: int
50     qualified_teacher_ids: tuple = ()
51     required_equipment: tuple = ()
52     allowed_slots: tuple = ()
53
54
55 @dataclass(frozen=True)
56 class Problem:
57     time_slots: tuple
58     teachers: tuple
59     rooms: tuple
60     classes: tuple
61     courses: tuple
```

```
62
63
64 @dataclass(frozen=True)
65 class ScheduledLesson:
66     lesson_id: str
67     course_id: str
68     course_name: str
69     class_id: str
70     class_name: str
71     teacher_id: str
72     teacher_name: str
73     room_id: str
74     room_name: str
75     slot_id: str
76     day: str
77     period: str
78
79     def to_dict(self):
80         return asdict(self)
81
82
83 @dataclass
84 class SolveResult:
85     status: str
86     schedule: list = field(default_factory=list)
87     conflicts: list = field(default_factory=list)
88     metrics: dict = field(default_factory=dict)
89
90     def to_dict(self):
91         return {
92             "status": self.status,
93             "metrics": self.metrics,
94             "conflicts": self.conflicts,
95             "schedule": [item.to_dict() for item in self.schedule],
96         }
```

附录 B：演示数据示例（节选）

```
1 // data/demo.json 完整内容（节选）
2 {
3     "time_slots": [
```



```

4      {"id": "MON_0800", "day": "周一", "period": "08:00-09:30", "order": 1},
5      {"id": "MON_1000", "day": "周一", "period": "10:00-11:30", "order": 2},
6      {"id": "MON_1330", "day": "周一", "period": "13:30-15:00", "order": 3},
7      {"id": "MON_1530", "day": "周一", "period": "15:30-17:00", "order": 4},
8      {"id": "FRI_1530", "day": "周五", "period": "15:30-17:00", "order": 20}
9  ],
10  "teachers": [
11      {"id": "T01", "name": "张老师", "qualifications": ["数学"],
12          "unavailable": ["WED_1000"],
13          "preferred_slots": ["MON_0800", "TUE_0800", "THU_1000"]},
14      {"id": "T02", "name": "李老师", "qualifications": ["数学", "物理"],
15          "unavailable": ["TUE_1000"],
16          "preferred_slots": ["MON_1330", "WED_1330", "FRI_0800"]},
17      {"id": "T03", "name": "王老师", "qualifications": ["英语"],
18          "unavailable": ["FRI_1530"],
19          "preferred_slots": ["TUE_1000", "THU_1330", "FRI_1000"]},
20      {"id": "T04", "name": "赵老师", "qualifications": ["物理"],
21          "unavailable": ["MON_0800"],
22          "preferred_slots": ["MON_1330", "WED_1330", "FRI_0800"]}
23  ],
24  "rooms": [
25      {"id": "R101", "name": "思源教室", "campus": "鼓楼校区",
26          "capacity": 30, "equipment": ["投影", "白板"]},
27      {"id": "R201", "name": "致远教室", "campus": "鼓楼校区",
28          "capacity": 45, "equipment": ["投影", "白板", "实验台"],
29          "unavailable": ["FRI_1330"]},
30      {"id": "R301", "name": "博雅教室", "campus": "仙林校区",
31          "capacity": 25, "equipment": ["投影", "白板"]}
32  ],
33  "classes": [
34      {"id": "C01", "name": "高一数学A班", "size": 28,
35          "preferred_slots": ["MON_0800", "WED_1000", "FRI_1330"]},
36      {"id": "C02", "name": "高一英语A班", "size": 22,
37          "unavailable": ["MON_0800"],
38          "preferred_slots": ["TUE_1000", "THU_1330", "FRI_1000"]},
39      {"id": "C03", "name": "高二物理A班", "size": 35,
40          "preferred_slots": ["MON_1330", "WED_1330", "FRI_0800"]},
41      {"id": "C04", "name": "初三数学冲刺班", "size": 24,
42          "unavailable": ["FRI_1530", "THU_1530"],
43          "preferred_slots": ["TUE_0800", "THU_1000", "FRI_1000"]}

```

```
44 ],
45 "courses": [
46   {"id": "CR01", "name": "高一数学提升", "subject": "数学",
47     "class_id": "C01", "sessions": 2,
48     "qualified_teacher_ids": ["T01", "T02"],
49     "required_equipment": ["投影"],
50     "allowed_slots": ["MON_0800", "WED_1000", "FRI_1330"]},
51   {"id": "CR02", "name": "高一英语阅读", "subject": "英语",
52     "class_id": "C02", "sessions": 2,
53     "qualified_teacher_ids": ["T03"],
54     "required_equipment": ["投影"],
55     "allowed_slots": ["TUE_1000", "THU_1330", "FRI_1000"]},
56   {"id": "CR03", "name": "高二物理实验", "subject": "物理",
57     "class_id": "C03", "sessions": 2,
58     "qualified_teacher_ids": ["T02", "T04"],
59     "required_equipment": ["实验台"],
60     "allowed_slots": ["MON_1330", "WED_1330", "FRI_0800"]},
61   {"id": "CR04", "name": "初三数学冲刺", "subject": "数学",
62     "class_id": "C04", "sessions": 2,
63     "qualified_teacher_ids": ["T01", "T02"],
64     "required_equipment": ["白板"],
65     "allowed_slots": ["TUE_0800", "THU_1000", "FRI_1000"]}
66 ]
67 }
```

附录 C：API 接口详细定义

C.1 GET /api/health

用途：健康检查。请求：无。

```
1 {
2   "status": "ok",
3   "service": "ClassMind",
4   "solver": "OR-Tools CP-SAT"
5 }
```

C.2 GET /api/plans

用途：获取 3 套候选方案。请求：无。

```
1 {
2   "plans": [
3     {
```

```
4      "id": "student",
5      "name": "学生体验优先",
6      "status": "OPTIMAL",
7      "metrics": {
8          "solve_time_ms": 38.7,
9          "lesson_count": 8,
10         "hard_conflict_count": 0,
11         "teacher_count": 4,
12         "room_count": 3,
13         "strategy": "student",
14         "objective_value": 137.0
15     },
16     "conflicts": [],
17     "schedule": [ "...8 条课次..." ],
18     "scorecard": {
19         "hard_conflicts": 0,
20         "student_preference_rate": 100.0,
21         "teacher_preference_rate": 75.0,
22         "capacity_fit_rate": 90.3,
23         "stability_variance": 0.45,
24         "overall_score": 89.1
25     }
26 }
27 ]
28 }
```

C.3 GET /api/dashboard

用途：获取决策驾驶舱所需数据。**响应：**同单个 SolveResult，额外含 teacher_load 字段。

C.4 POST /api/solve

用途：对自定义业务数据求解。**请求体：**与 data/demo.json 同结构，可选 strategy 字段。**响应：**单个 SolveResult。

C.5 POST /api/reschedule

用途：教师请假最小影响调课。

```
1 { "teacher_id": "T01", "slot_id": "MON_0800" }
```

```
1 {
2     "request": {"type": "teacher_leave", "teacher_id": "T01",
3               "slot_id": "MON_0800"},

```

```
4  "before": { "status":"OPTIMAL", "schedule":[ "... " ] },
5  "after": { "status":"OPTIMAL", "schedule":[ "... " ] },
6  "diff": {
7    "change_count": 2,
8    "changes": [
9      {
10       "lesson_id": "CR01-1",
11       "before": {"teacher_id":"T01","room_id":"R101",
12                 "slot_id":"MON_0800"},
13       "after": {"teacher_id":"T02","room_id":"R101",
14                 "slot_id":"MON_0800"}
15     }
16   ]
17 },
18 "impact": {
19   "changed_lessons": 2,
20   "affected_classes": ["高一数学A班"],
21   "affected_class_count": 1
22 }
23 }
```

附录 D：完整源码

完整源码托管在 GitHub:

github.com/Lenny-lab/classmind-scheduler

本附录给出 solver.py 核心算法完整代码以供评审查阅，其余源码（api.py / service.py / validator.py / io.py / models.py / cli.py）请参见上述仓库的 classmind/ 目录。

```
1  # classmind/solver.py 关键算法
2  from ortools.sat.python import cp_model
3
4
5  STRATEGY_WEIGHTS = {
6    "student": {"preference": 10, "stability": 2,
7               "efficiency": 1, "change": 0, "compactness": 3},
8    "teacher": {"preference": 2, "stability": 10,
9               "efficiency": 1, "change": 0, "compactness": 3},
10   "efficiency": {"preference": 1, "stability": 2,
11                 "efficiency": 10, "change": 0, "compactness": 3},
```

```

12     "balanced": {"preference": 5, "stability": 5,
13                 "efficiency": 5, "change": 0, "compactness": 3},
14     "reschedule": {"preference": 1, "stability": 1,
15                   "efficiency": 1, "change": 100, "compactness": 0},
16 }
17
18
19 def _per_lesson_soft_costs(problem, variables, weights,
20                           baseline_by_lesson, strategy):
21     teachers = {x.id: x for x in problem.teachers}
22     rooms = {x.id: x for x in problem.rooms}
23     classes = {x.id: x for x in problem.classes}
24     slots = {x.id: x for x in problem.time_slots}
25
26     costs = []
27     for (lesson_id, teacher_id, room_id, slot_id), variable in variables.items():
28         :
29         course = next(x for x in problem.courses
30                       if x.id == lesson_id.rsplit("-", 1)[0])
31         group = classes[course.class_id]
32
33         group_hit = int(bool(group.preferred_slots)
34                        and slot_id not in group.preferred_slots)
35         teacher_hit = int(bool(teachers[teacher_id].preferred_slots)
36                           and slot_id not in teachers[teacher_id].
37                           preferred_slots)
38         preference_cost = group_hit + teacher_hit
39
40
41         capacity = max(1, rooms[room_id].capacity)
42         capacity_waste = max(0, capacity - group.size) / capacity
43
44         max_order = max(1, max(s.order for s in problem.time_slots))
45         compactness_cost = (slots[slot_id].order - 1) / max_order
46
47         baseline = baseline_by_lesson.get(lesson_id)
48         change_cost = 1 if (strategy == "reschedule"
49                           and baseline != (teacher_id, room_id, slot_id)) else
50         0
51
52     cost_expr = (

```

```
49         weights["preference"] * preference_cost
50         + weights["efficiency"] * capacity_waste
51         + weights["compactness"] * compactness_cost
52         + weights["change"] * change_cost
53     )
54     costs.append((cost_expr, variable))
55 return costs
56
57
58 def _stability_cost(problem, variables, weights, n_slots, model):
59     if weights["stability"] == 0 or n_slots == 0:
60         return 0, 0
61
62     teachers = list(problem.teachers)
63     days = sorted({s.day for s in problem.time_slots})
64     per_day_teacher_count = defaultdict(list)
65     for (lesson_id, teacher_id, room_id, slot_id), variable in variables.items():
66         :
67         day = next(s.day for s in problem.time_slots if s.id == slot_id)
68         per_day_teacher_count[(teacher_id, day)].append(variable)
69
70     penalty_expr = []
71     expected_per_day = max(1, n_slots // max(1, len(teachers) * len(days)))
72
73     for teacher in teachers:
74         for day in days:
75             exprs = per_day_teacher_count.get((teacher.id, day), [])
76             if not exprs:
77                 continue
78             daily_count = sum(exprs)
79             dev_pos = model.new_int_var(0, len(exprs),
80                                     f"dev_pos_{teacher.id}_{day}")
81             dev_neg = model.new_int_var(0, len(exprs),
82                                     f"dev_neg_{teacher.id}_{day}")
83             model.add(dev_pos >= daily_count - expected_per_day)
84             model.add(dev_neg >= expected_per_day - daily_count)
85             penalty_expr.append((dev_pos + dev_neg))
86
87     if not penalty_expr:
88         return 0, 0
```

```
88     return 1, weights["stability"] * sum(penalty_expr)
89
90
91 def solve_schedule(problem, time_limit_seconds=15.0,
92                     strategy="balanced", baseline=None):
93     data_errors = validate_problem(problem)
94     if data_errors:
95         return SolveResult(status="INVALID_DATA", conflicts=data_errors)
96
97     if strategy not in STRATEGY_WEIGHTS:
98         return SolveResult(status="INVALID_DATA",
99                             conflicts=[f"未知策略: {strategy}"])
100     weights = STRATEGY_WEIGHTS[strategy]
101
102     teachers = {x.id: x for x in problem.teachers}
103     rooms = {x.id: x for x in problem.rooms}
104     classes = {x.id: x for x in problem.classes}
105     slots = {x.id: x for x in problem.time_slots}
106     slot_ids = [x.id for x in sorted(problem.time_slots,
107                                     key=lambda s: s.order)]
108
109     lesson_specs = [
110         (f"{course.id}-{session + 1}", course)
111         for course in problem.courses
112         for session in range(course.sessions)
113     ]
114     model = cp_model.CpModel()
115     variables = {}
116     lesson_vars = defaultdict(list)
117
118     for lesson_id, course in lesson_specs:
119         group = classes[course.class_id]
120         allowed = set(course.allowed_slots or slot_ids)
121         for teacher_id in course.qualified_teacher_ids:
122             teacher = teachers[teacher_id]
123             if course.subject not in teacher.qualifications:
124                 continue
125             for room in problem.rooms:
126                 if (room.capacity < group.size
127                     or not set(course.required_equipment)
```

```

128         .issubset(room.equipment)):
129             continue
130         for slot_id in slot_ids:
131             if slot_id not in allowed:
132                 continue
133             if slot_id in teacher.unavailable:
134                 continue
135             if slot_id in room.unavailable:
136                 continue
137             if slot_id in group.unavailable:
138                 continue
139             key = (lesson_id, teacher_id, room.id, slot_id)
140             variables[key] = model.new_bool_var(
141                 "x_" + "_".join(key))
142             lesson_vars[lesson_id].append(variables[key])
143
144     impossible = [lid for lid, _ in lesson_specs if not lesson_vars[lid]]
145     if impossible:
146         return SolveResult(status="INFEASIBLE",
147                             conflicts=[f"课次无任何可行: {x}"
148                                         for x in impossible])
149
150     for lesson_id, _ in lesson_specs:
151         model.add_exactly_one(lesson_vars[lesson_id])
152     for teacher_id in teachers:
153         for slot_id in slot_ids:
154             model.add_at_most_one(
155                 [v for (l, t, r, s), v in variables.items()
156                  if t == teacher_id and s == slot_id])
157     for room_id in rooms:
158         for slot_id in slot_ids:
159             model.add_at_most_one(
160                 [v for (l, t, r, s), v in variables.items()
161                  if r == room_id and s == slot_id])
162     for class_id in classes:
163         course_ids = {x.id for x in problem.courses
164                       if x.class_id == class_id}
165         for slot_id in slot_ids:
166             model.add_at_most_one(
167                 [v for (l, t, r, s), v in variables.items()

```



```

168         if l.rsplit("-", 1)[0] in course_ids and s == slot_id])
169     for course in problem.courses:
170         lesson_ids = [f"{course.id}-{i + 1}"
171                        for i in range(course.sessions)]
172     for slot_id in slot_ids:
173         model.add_at_most_one(
174             [v for (l, t, r, s), v in variables.items()
175              if l in lesson_ids and s == slot_id])
176
177     baseline_by_lesson = {x.lesson_id: (x.teacher_id, x.room_id, x.slot_id)
178                            for x in (baseline or [])}
179     per_lesson = _per_lesson_soft_costs(problem, variables, weights,
180                                         baseline_by_lesson, strategy)
181     objective_terms = [expr * var for expr, var in per_lesson]
182     stability_active, stability_expr = _stability_cost(
183         problem, variables, weights, len(lesson_specs), model)
184     if stability_active:
185         objective_terms.append(stability_expr)
186     model.minimize(sum(objective_terms))
187
188     solver = cp_model.CpSolver()
189     solver.parameters.max_time_in_seconds = time_limit_seconds
190     solver.parameters.num_search_workers = 8
191     solver.parameters.random_seed = 42
192     started = perf_counter()
193     status_code = solver.solve(model)
194     elapsed_ms = round((perf_counter() - started) * 1000, 2)
195     if status_code not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
196         return SolveResult(status="INFEASIBLE",
197                             conflicts=["当前硬约束组合无可行课表"],
198                             metrics={"solve_time_ms": elapsed_ms})
199
200     courses = {x.id: x for x in problem.courses}
201     schedule = []
202     for (lesson_id, teacher_id, room_id, slot_id), variable in variables.items():
203         if solver.value(variable) != 1:
204             continue
205         course = courses[lesson_id.rsplit("-", 1)[0]]
206         teacher = teachers[teacher_id]

```

```

207     room = rooms[room_id]
208     slot = slots[slot_id]
209     group = classes[course.class_id]
210     schedule.append(ScheduledLesson(
211         lesson_id=lesson_id, course_id=course.id,
212         course_name=course.name,
213         class_id=group.id, class_name=group.name,
214         teacher_id=teacher.id, teacher_name=teacher.name,
215         room_id=room.id, room_name=room.name,
216         slot_id=slot.id, day=slot.day, period=slot.period,
217     ))
218     schedule.sort(key=lambda x: (slots[x.slot_id].order,
219                                 x.room_id, x.lesson_id))
220     conflicts = validate_schedule(problem, schedule)
221     return SolveResult(
222         status="OPTIMAL" if status_code == cp_model.OPTIMAL else "FEASIBLE",
223         schedule=schedule,
224         conflicts=conflicts,
225         metrics={
226             "solve_time_ms": elapsed_ms,
227             "lesson_count": len(schedule),
228             "hard_conflict_count": len(conflicts),
229             "teacher_count": len({x.teacher_id for x in schedule}),
230             "room_count": len({x.room_id for x in schedule}),
231             "strategy": strategy,
232             "objective_value": round(solver.objective_value, 2),
233         },
234     )
235
236
237 def solve_portfolio(problem, time_limit_seconds=15.0):
238     return {
239         strategy: solve_schedule(problem, time_limit_seconds, strategy)
240         for strategy in ("student", "teacher", "efficiency")
241     }

```

附录 E：术语表

表 20 术语表

术语	说明
CP-SAT	Constraint Programming - SAT，Google OR-Tools 中的混合求解引擎，结合了约束传播、SAT 冲突学习与线性规划松弛。
CP	Constraint Programming，约束规划。
CSP	Constraint Satisfaction Problem，约束满足问题。
SAT	Boolean Satisfiability Problem，布尔可满足性问题。
OR-Tools	Google 开源的运筹学工具包，Apache 2.0 协议。
硬约束	必须严格满足的约束，违反则方案无效（如”教师同 0 冲突”）。
软目标	可以被违反但应尽量满足的目标（如”学生偏好尽量命中”）。
策略权重	不同维度的软目标在总目标中的相对重要性（0–10）。
策略	一组策略权重的命名（balanced / student / teacher / efficiency / reschedule）。
稳定性方差	教师日均课次的总体方差，越低表示越均匀。
调课	在原方案基础上做最小影响的修改。
决策变量	求解器可以控制的变量（如 $x_{l,t,r,s}$ ）。
预筛选	在生成决策变量前就剔除明显不可行的组合。
辅助变量	求解器自动引入的额外变量，用于表达复杂约束（如 dev_pos，dev_neg）。
归约	Reduction，把非线性目标近似为线性目标（如一阶泰勒）。
教务老师	K12 培训机构中负责排课的老师。
驾驶舱	Dashboard，集中展示关键指标的页面。

参考文献

1. Burke E K, Petrovic S. Recent research directions in automated timetabling[J]. European Journal of Operational Research, 2002, 140(2): 266–280.

2. de Werra D. Scheduling in sports[M]//Studies in Integer Programming. North-Holland, 1977: 381–395.

3. Google OR-Tools Team. OR-Tools: Software for Combinatorial Optimization[EB/OL]. <https://developers.google.com/optimization>, 2024.

4. Rossi F, van Beek P, Walsh T. Handbook of Constraint Programming[M]. Elsevier, 2006.

5. Apt K. Principles of Constraint Programming[M]. Cambridge University Press, 2003.

6. Laborie P, Rogerie J, Shaw P, et al. Reasoning with conditional time-intervals[C]//Proceedings of the Twenty-First International Florida Artificial Intelligence Research Society Conference. 2008: 555–560.

7. 中国教育部. 关于进一步减轻义务教育阶段学生作业负担和校外培训负担的意见 [Z]. 2021.

8. 艾瑞咨询. 2025 年中国 K12 教育行业研究报告 [R]. 上海: 艾瑞咨询, 2025.
9. 中国教育学会. 中小学课外辅导行业白皮书 2024[R]. 北京: 中国教育学会, 2024.
10. 蒋望东, 林培群. 基于约束规划的高校本专科排课系统 [J]. 计算机应用与软件, 2015, 32(6): 95–99.
11. 卢晓晴, 周戈. 基于遗传算法的高中走班排课优化研究 [J]. 教育信息化, 2020(8): 67–72.
12. 钟珞, 童莹. 一对一辅导机构排课管理系统的设计与实现 [J]. 计算机工程与设计, 2019, 40(11): 3321–3327.
13. 字节跳动飞书团队. 飞书开放平台开发者文档 [EB/OL]. <https://open.feishu.cn/document>, 2024.
14. 飞书多维表格技术白皮书 [Z]. 北京字节跳动科技有限公司, 2024.
15. Cormen T H, Leiserson C E, Rivest R L, et al. Introduction to Algorithms (4th ed.)[M]. MIT Press, 2022.
16. Taha H A. Operations Research: An Introduction (10th ed.)[M]. Pearson, 2017.
17. Python Software Foundation. The Python Language Reference (3.12)[EB/OL]. <https://docs.python.org/3.12/>, 2024.
18. 维基百科. Constraint Programming[EB/OL]. https://en.wikipedia.org/wiki/Constraint_programming, 2024.
19. Wainwright R L. Introduction to Expert Systems[M]. Macmillan, 1990.
20. Russell S, Norvig P. Artificial Intelligence: A Modern Approach (4th ed.)[M]. Pearson, 2020.

致 谢

本项目能够顺利完成，离不开多方面的支持与帮助，在此一并致谢：

- 感谢 Google OR-Tools 团队开源的 CP-SAT 求解器，这是本项目最核心的依赖；
- 感谢 Python 软件基金会维护的 Python 3.12 解释器、标准库 `unittest` / `http.server` / `dataclasses` / `json` 等模块；
- 感谢 GitHub 提供免费的 Actions 持续集成服务；
- 感谢 12 家 K12 机构的教务老师在调研阶段的耐心配合；
- 感谢飞书高校应用创新大赛组委会提供展示平台；

- 感谢指导老师在选题、调研、答辩各环节的悉心指导；
- 感谢团队成员在 6 周时间内的紧密协作与持续投入。

由于作者水平所限，文中难免存在不足之处，敬请各位老师与读者批评指正。

李帅锋 敬上

2026 年 7 月 12 日